3. Use a static or instance method handler and add the XML comments, as shown in the following listing.

Listing 11.11 Adding documentation comments to an endpoint handler

```
using Microsoft.AspNetCore.Mvc;
using System.Collections.Concurrent;
using System.Reflection;

WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(opts =>                                  ❶
{
    var file = $"{Assembly.GetExecutingAssembly().GetName().Name}.xml";
    opts.IncludeXmlComments(Path.Combine(AppContext.BaseDirectory, file));
});

WebApplication app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

var _fruit = new ConcurrentDictionary<string, Fruit>();

var handler = new FruitHandler(fruit);                                  ❷
app.MapGet("/fruit/{id}", handler.GetFruit)                             ❷
    .WithName("GetFruit");                                              ❸

app.Run();
record Fruit(string Name, int Stock);

internal class FruitHandler
{
    private readonly ConcurrentDictionary<string, Fruit> _fruit;
    public FruitHandler(ConcurrentDictionary<string, Fruit> fruit)
    {
        _fruit = fruit;
    }

    /// <summary>
    /// Fetches a fruit by id, or returns 404 if it does not exist         ❹
    /// </summary>                                                        ❹
    /// <param name="id" >The ID of the fruit to fetch</param>            ❹
    /// <response code="200">Returns the fruit if it exists</response>    ❹
    /// <response code="404">If the fruit doesn't exist</response>        ❹
    [ProducesResponseType(typeof(Fruit), 200)]                            ❺
    [ProducesResponseType(typeof(HttpValidationProblemDetails),           ❺
        404, "application/problem+json")]                                 ❺
```